

AGH

AKADEMIA GÓRNICZO-HUTNICZA
IM. STANISŁAWA STASZICA
W KRAKOWIE

Basketball League Database

Bazy Danych 2024/2025

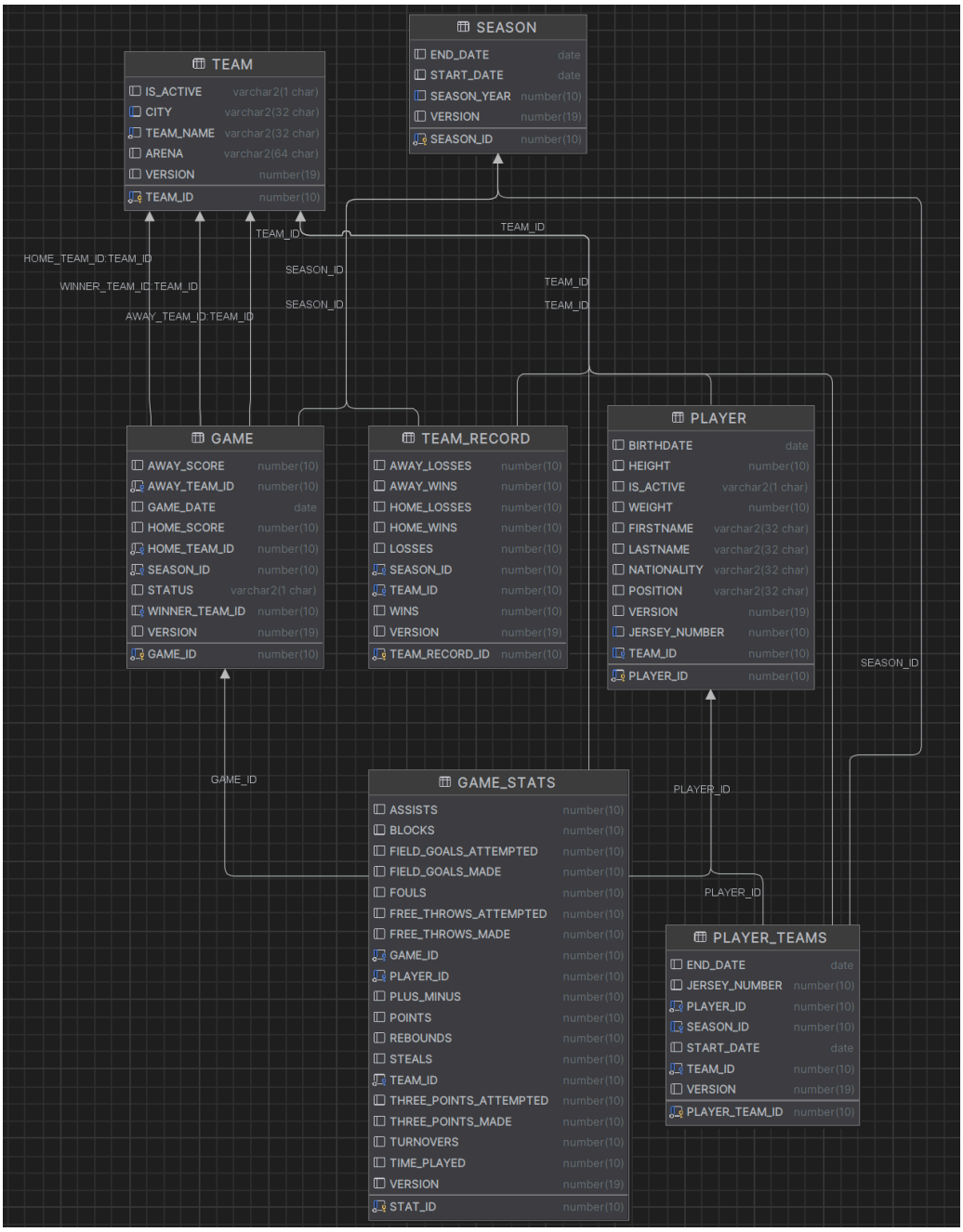
Adam Głowacki, Jan Sarba

1. Wprowadzenie

Celem projektu było zaprojektowanie i stworzenie aplikacji do zarządzania ligą koszykówki. W ramach realizacji zbudowano relacyjną bazę danych w systemie Oracle, która umożliwia przechowywanie informacji o drużynach, zawodnikach, meczach oraz wynikach.

Do komunikacji z bazą danych wykorzystano technologię Hibernate (implementacja JPA), co umożliwiło mapowanie obiektowo-relacyjne (ORM). Logika aplikacji oraz interfejs programistyczny (REST API) zostały zaimplementowane w oparciu o Spring Boot.

2. Schemat bazy danych



3. Encje

W Hibernate operujemy na encjach – czyli klasach Javy, które odwzorowują tabele w relacyjnej bazie danych. Każda encja jest oznaczona adnotacją Entity.

Relacje między encjami, takie jak OneToMany czy ManyToOne, umożliwiają odwzorowanie powiązań między tabelami bez konieczności pisania ręcznych zapytań SQL. Struktura klas encji jest ściśle dopasowana do struktury relacyjnej bazy danych, a dane mogą być automatycznie walidowane przy użyciu adnotacji takich jak NotNull, Min, Max.

Dodatkowo, w encjach zastosowano adnotację Version, która umożliwia mechanizm optymistycznej kontroli współbieżności. Dzięki temu każda transakcja modyfikująca dane sprawdza, czy inna nie zmieniła ich wcześniej – co zabezpiecza integralność danych.

Encje tworzone są w sposób spójny i zgodny ze standardami JPA, co ułatwia zarządzanie schematem bazy oraz integrację z innymi warstwami aplikacji (service, repository).

3.1 Team

Encja Team reprezentuje drużynę koszykarską i przechowuje jej podstawowe informacje, takie jak nazwa, miasto, nazwa hali oraz status aktywności.

```
@Check(constraints = "IS_ACTIVE IN ('Y','N')")
@Entity
@Table(name = "TEAM", uniqueConstraints = {
        @UniqueConstraint(columnNames = {"TEAM_NAME", "CITY"})
})
public class Team {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "team_seq")
    @SequenceGenerator(name = "team_seq", sequenceName = "TEAM_SEQ", allocationSize = 1)
    @Column(name = "TEAM_ID")
    private Integer id;

    @NotNull
    @Column(name = "TEAM_NAME", length = 32, nullable = false)
    private String name;

    @NotNull
    @Column(name = "CITY", length = 32, nullable = false)
    private String city;

    @Column(name = "ARENA", length = 64)
    private String arena;

    @Pattern(regexp = "[YN]")
    @NotNull
    @Column(name = "IS_ACTIVE", length = 1, nullable = false)
    private String isActive;

    @OneToMany(mappedBy = "team", cascade = CascadeType.REMOVE, orphanRemoval = true)
    private List<GameStats> gameStats;

    @OneToMany(mappedBy = "team", cascade = CascadeType.REMOVE, orphanRemoval = true)
    private List<PlayerTeam> playerTeams;

    @OneToMany(mappedBy = "team", cascade = CascadeType.REMOVE, orphanRemoval = true)
    private List<TeamRecord> teamRecords;

    @OneToMany(mappedBy = "team")
    private List<Player> players;

    @Version
    @Column(name = "VERSION")
    private Long version = 0L;

    public Team() {
    }

    public Team(String name, String city, String arena, String isActive) {
        this.name = name;
        this.city = city;
        this.arena = arena;
        this.isActive = isActive;
    }
```

```
    public Integer getId() {
        return id;
    }
    public void setId(Integer id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getCity() {
        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }
    public String getArena() {
        return arena;
    }
    public void setArena(String arena) {
        this.arena = arena;
    }
    public String getIsActive() {
        return isActive;
    }
    public void setIsActive(String isActive) {
        this.isActive = isActive;
    }
    public List<Player> getPlayers() { return players; }
    public void setPlayers(List<Player> players) { this.players = players; }
    public Long getVersion() { return version; }
    public void setVersion(Long version) {
        this.version = version;
    }
}
```

3.2 TeamRecord

Encja TeamRecord reprezentuje bilans drużyny w danym sezonie, przechowując liczbę zwycięstw i porażek – zarówno ogólnych, jak i podzielonych na mecze domowe i wyjazdowe.

```
@Check(constraints = "WINS + LOSSES = HOME_WINS + HOME_LOSSES + AWAY_WINS + AWAY_LOSSES")
@Entity
@Table(name = "TEAM_RECORD",
        uniqueConstraints = @UniqueConstraint(columnNames = {"TEAM_ID", "SEASON_ID"}))
public class TeamRecord {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "team_record_seq")
    @SequenceGenerator(name = "team_record_seq", sequenceName = "TEAM_RECORD_SEQ", allocationSize = 1)
    @Column(name = "TEAM_RECORD_ID")
    public Integer id;

    @NotNull
    @ManyToOne
    @JoinColumn(name = "TEAM_ID", nullable = false)
    public Team team;

    @NotNull
    @ManyToOne
    @JoinColumn(name = "SEASON_ID", nullable = false)
    public Season season;

    @Min(0)
    @NotNull
    @Column(name = "WINS", nullable = false)
    public Integer wins = 0;

    @Min(0)
    @NotNull
    @Column(name = "LOSSES", nullable = false)
    public Integer losses = 0;

    @Min(0)
    @NotNull
    @Column(name = "HOME_WINS", nullable = false)
    public Integer homeWins = 0;

    @Min(0)
    @NotNull
```

```
    @Column(name = "HOME_LOSSES", nullable = false)
    public Integer homeLosses = 0;

    @Min(0)
    @NotNull
    @Column(name = "AWAY_WINS", nullable = false)
    public Integer awayWins = 0;

    @Min(0)
    @NotNull
    @Column(name = "AWAY_LOSSES", nullable = false)
    public Integer awayLosses = 0;

    @Version
    @Column(name = "VERSION")
    private Long version = 0L;

    public TeamRecord() {
    }

    public TeamRecord(Team team, Season season, Integer wins, Integer losses,
                      Integer homeWins, Integer homeLosses,
                      Integer awayWins, Integer awayLosses) {
        this.team = team;
        this.season = season;
        this.wins = wins;
        this.losses = losses;
        this.homeWins = homeWins;
        this.homeLosses = homeLosses;
        this.awayWins = awayWins;
        this.awayLosses = awayLosses;
    }
    public void incrementWins() { this.wins++; }
    public void incrementLosses() { this.losses++; }
    public void incrementHomeWins() { this.homeWins++; }
    public void incrementHomeLosses() { this.homeLosses++; }
    public void incrementAwayWins() { this.awayWins++; }
    public void incrementAwayLosses() { this.awayLosses++; }
    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }
    public Team getTeam() { return team; }
    public void setTeam(Team team) { this.team = team; }
    public Season getSeason() { return season; }
    public void setSeason(Season season) { this.season = season; }
    public Integer getWins() { return wins; }
    public void setWins(Integer wins) { this.wins = wins; }
    public Integer getLosses() { return losses; }
    public void setLosses(Integer losses) { this.losses = losses; }
    public Integer getHomeWins() { return homeWins; }
    public void setHomeWins(Integer homeWins) { this.homeWins = homeWins; }
    public Integer getHomeLosses() { return homeLosses; }
    public void setHomeLosses(Integer homeLosses) { this.homeLosses = homeLosses; }
    public Integer getAwayWins() { return awayWins; }
    public void setAwayWins(Integer awayWins) { this.awayWins = awayWins; }
    public Integer getAwayLosses() { return awayLosses; }
    public void setAwayLosses(Integer awayLosses) { this.awayLosses = awayLosses; }
    public Long getVersion() {
        return version;
    }
    public void setVersion(Long version) {
        this.version = version;
    }
}
```

3.3 Season

Encja Season reprezentuje sezon rozgrywek i zawiera informacje takie jak rok sezonu, data rozpoczęcia i zakończenia oraz powiązane rekordy drużyn.

```
@Entity
@Table(name = "SEASON", uniqueConstraints = {
        @UniqueConstraint(columnNames = {"SEASON_YEAR"})
})
@Check(constraints = "START_DATE <= END_DATE")
public class Season {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "season_seq")
    @SequenceGenerator(name = "season_seq", sequenceName = "SEASON_SEQ", allocationSize = 1)
    @Column(name = "SEASON_ID")
    public Integer id;

    @NotNull
```

```
    @Column(name = "SEASON_YEAR", nullable = false)
    private Integer seasonYear;

    @NotNull
    @Column(name = "START_DATE", nullable = false, unique = true)
    private LocalDate startDate;

    @Column(name = "END_DATE")
    private LocalDate endDate;

    @OneToMany(mappedBy = "season", cascade = CascadeType.REMOVE, orphanRemoval = true)
    public List<TeamRecord> teamRecords;

    @OneToMany(mappedBy = "season", cascade = CascadeType.REMOVE, orphanRemoval = true)
    private List<Game> games;

    @Version
    @Column(name = "VERSION")
    private Long version = 0L;

    public Season() {
    }

    public Season(Integer seasonYear, LocalDate startDate, LocalDate endDate) {
        this.seasonYear = seasonYear;
        this.startDate = startDate;
        this.endDate = endDate;
    }

    public Integer getId() {
        return id;
    }
    public void setId(Integer id) {
        this.id = id;
    }
    public Integer getSeasonYear() {
        return seasonYear;
    }
    public void setSeasonYear(Integer seasonYear) {
        this.seasonYear = seasonYear;
    }
    public LocalDate getStartDate() {
        return startDate;
    }
    public void setStartDate(LocalDate startDate) {
        this.startDate = startDate;
    }
    public LocalDate getEndDate() {
        return endDate;
    }
    public void setEndDate(LocalDate endDate) {
        this.endDate = endDate;
    }
    public Long getVersion() {
        return version;
    }
    public void setVersion(Long version) {
        this.version = version;
    }
}
```

3.4 Player

Encja Player reprezentuje zawodnika drużyny i przechowuje jego dane osobowe, takie jak imię, nazwisko, data urodzenia, numer na koszulce oraz przynależność do zespołu.

```
@Entity
@Table(name = "PLAYER",
        uniqueConstraints = {
                @UniqueConstraint(columnNames = {"TEAM_ID", "JERSEY_NUMBER"})
        })
public class Player {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "player_seq")
    @SequenceGenerator(name = "player_seq", sequenceName = "PLAYER_SEQ", allocationSize = 1)
    @Column(name = "PLAYER_ID")
    private Integer id;

    @ManyToOne
    @JoinColumn(name = "TEAM_ID")
    private Team team;
```

```
@NotNull
@Column(name = "FIRSTNAME", length = 32, nullable = false)
private String firstName;

@NotNull
@Column(name = "LASTNAME", length = 32, nullable = false)
private String lastName;

@NotNull
@Past
@Column(name = "BIRTHDATE", nullable = false)
private LocalDate birthDate;

@NotNull
@Column(name = "POSITION", length = 16, nullable = false)
private String position;

@NotNull
@Column(name = "NATIONALITY", length = 32, nullable = false)
private String nationality;

@Min(0)
@Max(99)
@Column(name = "JERSEY_NUMBER")
private Integer jerseyNumber;

@Min(100)
@Max(250)
@NotNull
@Column(name = "HEIGHT", nullable = false)
private Integer height;

@Min(40)
@Max(160)
@NotNull
@Column(name = "WEIGHT", nullable = false)
private Integer weight;

@Pattern(regexp = "[YN]")
@NotNull
@Column(name = "IS_ACTIVE", length = 1, nullable = false)
private String isActive;

@OneToMany(mappedBy = "player", cascade = CascadeType.ALL, orphanRemoval = true)
private List<PlayerTeam> playerTeams = new ArrayList<>();

@OneToMany(mappedBy = "player", cascade = CascadeType.REMOVE, orphanRemoval = true)
private List<GameStats> gameStats;

@Version
@Column(name = "VERSION")
private Long version = 0L;

public Player() {
}

public Player(Team team, String firstName, String lastName, LocalDate birthDate,
              String position, String nationality, Integer jerseyNumber,
              Integer height, Integer weight, String isActive) {
    this.team = team;
    this.firstName = firstName;
    this.lastName = lastName;
    this.birthDate = birthDate;
    this.position = position;
    this.nationality = nationality;
    this.jerseyNumber = jerseyNumber;
    this.height = height;
    this.weight = weight;
    this.isActive = isActive;
}

public Integer getId() {
    return id;
}
public void setId(Integer id) {
    this.id = id;
}
public Team getTeam() {
    return team;
}
public void setTeam(Team team) {
    this.team = team;
}
public String getFirstName() {
    return firstName;
}
public void setFirstName(String firstName) {
    this.firstName = firstName;
}
```

```
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
    public LocalDate getBirthDate() {
        return birthDate;
    }
    public void setBirthDate(LocalDate birthDate) {
        this.birthDate = birthDate;
    }
    public String getPosition() {
        return position;
    }
    public void setPosition(String position) {
        this.position = position;
    }
    public String getNationality() {
        return nationality;
    }
    public void setNationality(String nationality) {
        this.nationality = nationality;
    }
    public Integer getJerseyNumber() {
        return jerseyNumber;
    }
    public void setJerseyNumber(Integer jerseyNumber) {
        this.jerseyNumber = jerseyNumber;
    }
    public Integer getHeight() {
        return height;
    }
    public void setHeight(Integer height) {
        this.height = height;
    }
    public Integer getWeight() {
        return weight;
    }
    public void setWeight(Integer weight) {
        this.weight = weight;
    }
    public String getIsActive() {
        return isActive;
    }
    public void setIsActive(String isActive) {
        this.isActive = isActive;
    }
    public List<PlayerTeam> getPlayerTeams() {
        return playerTeams;
    }
    public void setPlayerTeams(List<PlayerTeam> playerTeams) {
        this.playerTeams = playerTeams;
    }
    public Long getVersion() {
        return version;
    }
    public void setVersion(Long version) {
        this.version = version;
    }
}
```

3.5 PlayerTeams

Encja PlayerTeams reprezentuje powiązanie zawodnika z drużyną w danym sezonie, uwzględniając okres obowiązywania kontraktu oraz przypisanie do konkretnego zespołu.

```
@Entity
@Table(name = "PLAYER_TEAMS",
        uniqueConstraints = {
                @UniqueConstraint(columnNames = {"PLAYER_ID", "TEAM_ID", "SEASON_ID"})
        }
)
@Check(constraints = "START_DATE <= END_DATE")
public class PlayerTeam {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "player_team_seq")
    @SequenceGenerator(name = "player_team_seq", sequenceName = "PLAYER_TEAM_SEQ", allocationSize = 1)
    @Column(name = "PLAYER_TEAM_ID")
    public Integer id;

    @NotNull
```

```
    @ManyToOne
    @JoinColumn(name = "PLAYER_ID", nullable = false)
    public Player player;

    @NotNull
    @ManyToOne
    @JoinColumn(name = "TEAM_ID", nullable = false)
    public Team team;

    @NotNull
    @Column(name = "SEASON_ID", nullable = false)
    public Integer seasonId;

    @NotNull
    @Column(name = "JERSEY_NUMBER", nullable = false)
    public Integer jerseyNumber;

    @NotNull
    @Column(name = "START_DATE", nullable = false)
    public LocalDate startDate;

    @Column(name = "END_DATE")
    public LocalDate endDate;

    @Version
    @Column(name = "VERSION")
    private Long version = 0L;

    public PlayerTeam() {
    }

    public PlayerTeam(Player player, Team team, Integer seasonId, Integer jerseyNumber, LocalDate startDate, LocalDate endDate) {
        this.player = player;
        this.team = team;
        this.seasonId = seasonId;
        this.jerseyNumber = jerseyNumber;
        this.startDate = startDate;
        this.endDate = endDate;
    }

    public Integer getId() { return id; }
    public Player getPlayer() { return player; }
    public void setPlayer(Player player) { this.player = player; }
    public Team getTeam() { return team; }
    public void setTeam(Team team) { this.team = team; }
    public Integer getJerseyNumber() { return jerseyNumber; }
    public void setJerseyNumber(Integer jerseyNumber) { this.jerseyNumber = jerseyNumber; }
    public LocalDate getStartDate() { return startDate; }
    public void setStartDate(LocalDate startDate) { this.startDate = startDate; }
    public LocalDate getEndDate() { return endDate; }
    public void setEndDate(LocalDate endDate) { this.endDate = endDate; }
    public Integer getSeasonId() { return seasonId; }
    public void setSeasonId(Integer seasonId) { this.seasonId = seasonId; }
    public Long getVersion() {
        return version;
    }
    public void setVersion(Long version) {
        this.version = version;
    }
}
```

3.6 Game

Encja Game reprezentuje mecz rozegrany w ramach danego sezonu, zawierając informacje o drużynie gospodarzy i gości, wyniku spotkania, zwycięzcy oraz dacie rozegrania.

```
@Entity
@Table(name = "GAME")
@Check(constraints = "HOME_TEAM_ID != AWAY_TEAM_ID")
public class Game {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "game_seq")
    @SequenceGenerator(name = "game_seq", sequenceName = "GAME_SEQ", allocationSize = 1)
    @Column(name = "GAME_ID")
    public Integer id;

    @NotNull
    @ManyToOne
    @JoinColumn(name = "SEASON_ID", nullable = false)
    private Season season;

    @NotNull
```

```
    @ManyToOne
    @JoinColumn(name = "HOME_TEAM_ID", nullable = false)
    private Team homeTeam;

    @NotNull
    @ManyToOne
    @JoinColumn(name = "AWAY_TEAM_ID", nullable = false)
    private Team awayTeam;

    @ManyToOne
    @JoinColumn(name = "WINNER_TEAM_ID")
    private Team winnerTeam;

    @Column(name = "GAME_DATE")
    private LocalDateTime gameDate;

    @Min(0)
    @NotNull
    @Column(name = "HOME_SCORE", nullable = false)
    private Integer homeScore = 0;

    @Min(0)
    @NotNull
    @Column(name = "AWAY_SCORE", nullable = false)
    private Integer awayScore = 0;

    @NotNull
    @Pattern(regexp = "[SAF]") // scheduled, active, finished
    @Column(name = "STATUS", length = 1, nullable = false)
    private String status = "S";

    @OneToMany(mappedBy = "game", cascade = CascadeType.REMOVE, orphanRemoval = true)
    private List<GameStats> gameStats;

    @Version
    @Column(name = "VERSION")
    private Long version = 0L;

    public Game() {
    }

    public Game(Season season, Team homeTeam, Team awayTeam, Team winnerTeam,
                LocalDateTime gameDate, Integer homeScore, Integer awayScore,
                String status) {
        this.season = season;
        this.homeTeam = homeTeam;
        this.awayTeam = awayTeam;
        this.winnerTeam = winnerTeam;
        this.gameDate = gameDate;
        this.homeScore = homeScore;
        this.awayScore = awayScore;
        this.status = status;
    }

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }
    public Season getSeason() { return season; }
    public void setSeason(Season season) { this.season = season; }
    public Team getHomeTeam() { return homeTeam; }
    public void setHomeTeam(Team homeTeam) { this.homeTeam = homeTeam; }
    public Team getAwayTeam() { return awayTeam; }
    public void setAwayTeam(Team awayTeam) { this.awayTeam = awayTeam; }
    public Team getWinnerTeam() { return winnerTeam; }
    public void setWinnerTeam(Team winnerTeam) { this.winnerTeam = winnerTeam; }
    public LocalDateTime getGameDate() { return gameDate; }
    public void setGameDate(LocalDateTime gameDate) { this.gameDate = gameDate; }
    public Integer getHomeScore() { return homeScore; }
    public void setHomeScore(Integer homeScore) { this.homeScore = homeScore; }
    public Integer getAwayScore() { return awayScore; }
    public void setAwayScore(Integer awayScore) { this.awayScore = awayScore; }
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
    public List<GameStats> getGameStats() { return gameStats; }
    public void setGameStats(List<GameStats> gameStats) { this.gameStats = gameStats; }
    public Long getVersion() {
        return version;
    }
    public void setVersion(Long version) {
        this.version = version;
    }
}
```

3.7 GameStats

Encja GameStats reprezentuje indywidualne statystyki zawodnika w konkretnym meczu, przechowując informacje takie jak liczba celnych i oddanych rzutów, punktów, zbiórek czy asyst.

```
@Entity
@Table(name = "GAME_STATS",
        uniqueConstraints = {
                @UniqueConstraint(columnNames = {"GAME_ID", "PLAYER_ID"})
        })
@Check(constraints =
        "FIELD_GOALS_MADE <= FIELD_GOALS_ATTEMPTED AND " +
        "THREE_POINTS_MADE <= THREE_POINTS_ATTEMPTED AND " +
        "FREE_THROWS_MADE <= FREE_THROWS_ATTEMPTED"
)
public class GameStats {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "game_stats_seq")
    @SequenceGenerator(name = "game_stats_seq", sequenceName = "GAME_STATS_SEQ", allocationSize = 1)
    @Column(name = "STAT_ID")
    public Integer id;

    @NotNull
    @ManyToOne
    @JoinColumn(name = "GAME_ID", nullable = false)
    public Game game;

    @NotNull
    @ManyToOne
    @JoinColumn(name = "PLAYER_ID", nullable = false)
    public Player player;

    @NotNull
    @ManyToOne
    @JoinColumn(name = "TEAM_ID", nullable = false)
    public Team team;

    @Min(0)
    @NotNull
    @Column(name = "TIME_PLAYED", nullable = false)
    public Integer timePlayedInSeconds = 0;

    @Min(0)
    @NotNull
    @Column(name = "POINTS", nullable = false)
    public Integer points = 0;

    @Min(0)
    @NotNull
    @Column(name = "REBOUNDS", nullable = false)
    public Integer rebounds = 0;

    @Min(0)
    @NotNull
    @Column(name = "ASSISTS", nullable = false)
    public Integer assists = 0;

    @Min(0)
    @NotNull
    @Column(name = "STEALS", nullable = false)
    public Integer steals = 0;

    @Min(0)
    @NotNull
    @Column(name = "BLOCKS", nullable = false)
    public Integer blocks = 0;

    @Min(0)
    @NotNull
    @Column(name = "TURNOVERS", nullable = false)
    public Integer turnovers = 0;

    @Min(0)
    @NotNull
    @Column(name = "FIELD_GOALS_MADE", nullable = false)
    public Integer fieldGoalsMade = 0;

    @Min(0)
    @NotNull
    @Column(name = "FIELD_GOALS_ATTEMPTED", nullable = false)
    public Integer fieldGoalsAttempted = 0;

    @Min(0)
```

```
@NotNull
@Column(name = "THREE_POINTS_MADE", nullable = false)
public Integer threePointsMade = 0;

@Min(0)
@NotNull
@Column(name = "THREE_POINTS_ATTEMPTED", nullable = false)
public Integer threePointsAttempted = 0;

@Min(0)
@NotNull
@Column(name = "FREE_THROWS_MADE", nullable = false)
public Integer freeThrowsMade = 0;

@Min(0)
@NotNull
@Column(name = "FREE_THROWS_ATTEMPTED", nullable = false)
public Integer freeThrowsAttempted = 0;

@Min(0)
@Max(5)
@NotNull
@Column(name = "FOULS", nullable = false)
public Integer fouls = 0;

@Version
@Column(name = "VERSION")
private Long version = 0L;

public GameStats() {
}

public GameStats(Game game, Player player, Team team, Integer timePlayedInSeconds, Integer points,
                 Integer rebounds, Integer assists, Integer steals, Integer blocks, Integer turnovers,
                 Integer fieldGoalsMade, Integer fieldGoalsAttempted,
                 Integer threePointsMade, Integer threePointsAttempted,
                 Integer freeThrowsMade, Integer freeThrowsAttempted,
                 Integer fouls) {
    this.game = game;
    this.player = player;
    this.team = team;
    this.timePlayedInSeconds = timePlayedInSeconds;
    this.points = points;
    this.rebounds = rebounds;
    this.assists = assists;
    this.steals = steals;
    this.blocks = blocks;
    this.turnovers = turnovers;
    this.fieldGoalsMade = fieldGoalsMade;
    this.fieldGoalsAttempted = fieldGoalsAttempted;
    this.threePointsMade = threePointsMade;
    this.threePointsAttempted = threePointsAttempted;
    this.freeThrowsMade = freeThrowsMade;
    this.freeThrowsAttempted = freeThrowsAttempted;
    this.fouls = fouls;
}

public GameStats(Game game, Player player, Team team) {
    this.game = game;
    this.player = player;
    this.team = team;
}

public Integer getId() { return id; }
public void setId(Integer id) { this.id = id; }
public Game getGame() { return game; }
public void setGame(Game game) { this.game = game; }
public Player getPlayer() { return player; }
public void setPlayer(Player player) { this.player = player; }
public Team getTeam() { return team; }
public void setTeam(Team team) { this.team = team; }
public Integer getTimePlayedInSeconds() { return timePlayedInSeconds; }
public void setTimePlayedInSeconds(Integer timePlayedInSeconds) { this.timePlayedInSeconds = timePlayedInSeconds; }
public Integer getPoints() { return points; }
public void setPoints(Integer points) { this.points = points; }
public Integer getRebounds() { return rebounds; }
public void setRebounds(Integer rebounds) { this.rebounds = rebounds; }
public Integer getAssists() { return assists; }
public void setAssists(Integer assists) { this.assists = assists; }
public Integer getSteals() { return steals; }
public void setSteals(Integer steals) { this.steals = steals; }
public Integer getBlocks() { return blocks; }
public void setBlocks(Integer blocks) { this.blocks = blocks; }
public Integer getTurnovers() { return turnovers; }
public void setTurnovers(Integer turnovers) { this.turnovers = turnovers; }
public Integer getFieldGoalsMade() { return fieldGoalsMade; }
public void setFieldGoalsMade(Integer fieldGoalsMade) { this.fieldGoalsMade = fieldGoalsMade; }
public Integer getFieldGoalsAttempted() { return fieldGoalsAttempted; }
public void setFieldGoalsAttempted(Integer fieldGoalsAttempted) { this.fieldGoalsAttempted = fieldGoalsAttempted; }
```

```
    public Integer getThreePointsMade() { return threePointsMade; }
    public void setThreePointsMade(Integer threePointsMade) { this.threePointsMade = threePointsMade; }
    public Integer getThreePointsAttempted() { return threePointsAttempted; }
    public void setThreePointsAttempted(Integer threePointsAttempted) { this.threePointsAttempted = threePointsAttempted; }
    public Integer getFreeThrowsMade() { return freeThrowsMade; }
    public void setFreeThrowsMade(Integer freeThrowsMade) { this.freeThrowsMade = freeThrowsMade; }
    public Integer getFreeThrowsAttempted() { return freeThrowsAttempted; }
    public void setFreeThrowsAttempted(Integer freeThrowsAttempted) { this.freeThrowsAttempted = freeThrowsAttempted; }
    public Integer getFouls() { return fouls; }
    public void setFouls(Integer fouls) { this.fouls = fouls; }
    public Long getVersion() { return version; }
    public void setVersion(Long version) {
        this.version = version;
    }
}
```

3.8 PlayerStatsView

Encja PlayerStatsView reprezentuje widok w bazie danych, który agreguje statystyki zawodnika dla konkretnej drużyny i sezonu. Zawiera zarówno dane sumaryczne, średnie i statystykę zaawansowaną eval, będącą wskaźnikiem efektywności zawodnika. Rekordy są identyfikowane na podstawie złożonego klucza głównego, składającego się z trzech kolumn - player_id, team_id, season_id

```
public class PlayerStatsViewId implements Serializable {

    private Integer playerId;
    private Integer seasonId;
    private Integer teamId;

    public PlayerStatsViewId() {
    }

    public PlayerStatsViewId(Integer playerId, Integer seasonId, Integer teamId) {
        this.playerId = playerId;
        this.seasonId = seasonId;
        this.teamId = teamId;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof PlayerStatsViewId)) return false;
        PlayerStatsViewId that = (PlayerStatsViewId) o;
        return Objects.equals(playerId, that.playerId) &&
                Objects.equals(seasonId, that.seasonId) &&
                Objects.equals(teamId, that.teamId);
    }

    @Override
    public int hashCode() {
        return Objects.hash(playerId, seasonId, teamId);
    }
}

@Entity
@Immutable
@IdClass(PlayerStatsViewId.class)
@Table(name = "PLAYER_STATS_BY_SEASON_VIEW")
public class PlayerStatsView {

    @Id
    @Column(name = "PLAYER_ID")
    private Integer playerId;

    @Id
    @Column(name = "SEASON_ID")
    private Integer seasonId;

    @Id
    @Column(name = "TEAM_ID")
    private Integer teamId;

    @Column(name = "FIRSTNAME")
    private String firstName;

    @Column(name = "LASTNAME")
    private String lastName;

    @Column(name = "TEAM_NAME")
    private String teamName;

    @Column(name = "GAMES_PLAYED")
    private Integer gamesPlayed;

    @Column(name = "TOTAL_POINTS")
    private Integer totalPoints;

    @Column(name = "TOTAL_REBOUNDS")
    private Integer totalRebounds;

    @Column(name = "TOTAL_ASSISTS")
    private Integer totalAssists;

    @Column(name = "TOTAL_STEALS")
    private Integer totalSteals;

    @Column(name = "TOTAL_BLOCKS")
    private Integer totalBlocks;
```

```
    @Column(name = "TOTAL_FGA")
    private Integer totalFga;

    @Column(name = "TOTAL_FGM")
    private Integer totalFgm;

    @Column(name = "TOTAL_FTA")
    private Integer totalFta;

    @Column(name = "TOTAL_FTM")
    private Integer totalFtm;

    @Column(name = "TOTAL_TURNOVERS")
    private Integer totalTurnovers;

    @Column(name = "TOTAL_3PA")
    private Integer total3pa;

    @Column(name = "TOTAL_3PM")
    private Integer total3pm;

    @Column(name = "AVG_POINTS")
    private Double avgPoints;

    @Column(name = "AVG_REBOUNDS")
    private Double avgRebounds;

    @Column(name = "AVG_ASSISTS")
    private Double avgAssists;

    @Column(name = "AVG_EVAL")
    private Double avgEval;

    public Integer getPlayerId()       { return playerId; }
    public Integer getSeasonId()       { return seasonId; }
    public Integer getTeamId()         { return teamId; }
    public String getFirstName()       { return firstName; }
    public String getLastName()        { return lastName; }
    public String getTeamName()        { return teamName; }
    public Integer getGamesPlayed()    { return gamesPlayed; }
    public Integer getTotalPoints()    { return totalPoints; }
    public Integer getTotalRebounds()  { return totalRebounds; }
    public Integer getTotalAssists()   { return totalAssists; }
    public Integer getTotalSteals()    { return totalSteals; }
    public Integer getTotalBlocks()    { return totalBlocks; }
    public Integer getTotalFga()       { return totalFga; }
    public Integer getTotalFgm()       { return totalFgm; }
    public Integer getTotalFta()       { return totalFta; }
    public Integer getTotalFtm()       { return totalFtm; }
    public Integer getTotalTurnovers() { return totalTurnovers; }
    public Integer getTotal3pa()       { return total3pa; }
    public Integer getTotal3pm()       { return total3pm; }
    public Double getAvgPoints()       { return avgPoints; }
    public Double getAvgRebounds()     { return avgRebounds; }
    public Double getAvgAssists()      { return avgAssists; }
    public Double getAvgEval()         { return avgEval; }
}
```

Kod Oracle widoku:

```
create view PLAYER_STATS_BY_SEASON_VIEW as
SELECT
    p.PLAYER_ID,
    p.FIRSTNAME,
    p.LASTNAME,
    pt.SEASON_ID,
    pt.TEAM_ID,
    t.TEAM_NAME,
    COUNT(DISTINCT gs.GAME_ID) AS games_played,
    SUM(gs.POINTS) AS total_points,
    SUM(gs.REBOUNDS) AS total_rebounds,
    SUM(gs.ASSISTS) AS total_assists,
    SUM(gs.STEALS) AS total_steals,
    SUM(gs.BLOCKS) AS total_blocks,
    SUM(gs.FIELD_GOALS_ATTEMPTED) AS total_fga,
    SUM(gs.FIELD_GOALS_MADE) AS total_fgm,
    SUM(gs.FREE_THROWS_ATTEMPTED) AS total_fta,
    SUM(gs.FREE_THROWS_MADE) AS total_ftm,
    SUM(gs.TURNOVERS) AS total_turnovers,

    SUM(gs.THREE_POINTS_ATTEMPTED) AS total_3pa,
    SUM(gs.THREE_POINTS_MADE) AS total_3pm,

    ROUND(AVG(gs.POINTS), 2) AS avg_points,
    ROUND(AVG(gs.REBOUNDS), 2) AS avg_rebounds,
    ROUND(AVG(gs.ASSISTS), 2) AS avg_assists,
    ROUND(AVG(
                gs.POINTS + gs.REBOUNDS + gs.ASSISTS + gs.STEALS + gs.BLOCKS
                    - (gs.FIELD_GOALS_ATTEMPTED - gs.FIELD_GOALS_MADE)
                    - (gs.FREE_THROWS_ATTEMPTED - gs.FREE_THROWS_MADE)
                    - gs.TURNOVERS
        ), 2) AS avg_eval
FROM
    PLAYER p
        JOIN PLAYER_TEAMS pt ON p.PLAYER_ID = pt.PLAYER_ID
        JOIN TEAM t ON pt.TEAM_ID = t.TEAM_ID
        JOIN GAME_STATS gs ON gs.PLAYER_ID = p.PLAYER_ID AND gs.TEAM_ID = pt.TEAM_ID
        JOIN GAME g ON gs.GAME_ID = g.GAME_ID AND g.SEASON_ID = pt.SEASON_ID
GROUP BY
    p.PLAYER_ID, p.FIRSTNAME, p.LASTNAME, pt.SEASON_ID, pt.TEAM_ID, t.TEAM_NAME
```

4. Architektura Spring Boot do zarządzania bazą

Zastosowaliśmy trójwarstwową architekturę, w której każda warstwa ma swoją odpowiedzialność. Całość została zrealizowana przy użyciu frameworka Spring Boot, który zapewnia automatyczną konfigurację komponentów, wsparcie dla wstrzykiwania zależności oraz integrację z bazą danych za pomocą Spring Data JPA.

1. Controller - odpowiada za obsługę żądań HTTP i deleguje zadania do warstwy serwisowej,
2. Service - interfejs + implementacja, odpowiada za operacje związane z przetwarzaniem danych oraz ich pośrednią obsługę w systemie
3. Repository – komunikuje się z bazą danych. Opiera się na Spring Data JPA, dzięki czemu wystarczy zdefiniować interfejs dziedziczący po JpaRepository, a Spring automatycznie tworzy gotową implementację do obsługi operacji CRUD.

Nie wszystkie moduły zawierają pełny zestaw trzech warstw. W niektórych przypadkach zastosowano jedynie repozytorium lub repozytorium z serwisem – np. gdy moduł służy tylko do odczytu danych (np. widok statystyk) lub gdy logika jest bardzo prosta i nie wymaga kontrolera.

4.1 Moduł Team

Moduł Team odpowiada za obsługę drużyn koszykarskich w systemie. Umożliwia przeglądanie, dodawanie, edytowanie i usuwanie drużyn poprzez interfejs użytkownika.

4.1.1 Controller

```
@Controller
@RequestMapping("/teams")
public class TeamController {

    private final TeamService teamService;
    private final TeamRecordService teamRecordService;
    private final SeasonService seasonService;

    public TeamController(TeamService teamService, TeamRecordService teamRecordService, SeasonService seasonService) {
        this.teamService = teamService;
        this.teamRecordService = teamRecordService;
        this.seasonService = seasonService;
    }

    @GetMapping
    public String listTeams(Model model) {
        model.addAttribute("listTeams", teamService.getAllTeams());
        return "teams/list";
    }

    @GetMapping("/new")
    public String showNewTeamForm(Model model) {
        model.addAttribute("team", new Team());
        model.addAttribute("formTitle", "Add New Team");
        return "teams/form";
    }

    @GetMapping("/edit/{id}")
    public String showEditForm(@PathVariable Integer id, Model model) {
        Team team = teamService.getTeamById(id);
        model.addAttribute("team", team);
        model.addAttribute("formTitle", "Edit Team");
        return "teams/form";
    }

    @PostMapping("/save")
    public String saveTeam(@ModelAttribute("team") Team team) {
        teamService.saveTeam(team);
        return "redirect:/teams";
    }

    @PostMapping("/delete/{id}")
    public String deleteTeam(@PathVariable Integer id) {
        teamService.deleteTeam(id);
        return "redirect:/teams";
    }

    @GetMapping("/standings")
    public String showStandings(@RequestParam(value = "seasonId", required = false) Integer seasonId, Model model) {
        List<Season> allSeasons = seasonService.getAllSeasons();
```

```
        model.addAttribute("seasons", allSeasons);

        if (allSeasons.isEmpty()) {
            model.addAttribute("teamRecords", List.of());
            model.addAttribute("selectedSeasonId", null);
            return "teams/standings";
        }

        Integer selectedSeasonId = (seasonId != null)
                ? seasonId
                : seasonService.getCurrentSeasonId();

        List<TeamRecord> teamRecords = (selectedSeasonId != null)
                ? teamRecordService.getRecordsBySeasonIdOrderByWinsDesc(selectedSeasonId)
                : List.of();

        model.addAttribute("teamRecords", teamRecords);
        model.addAttribute("selectedSeasonId", selectedSeasonId);
        return "teams/standings";
    }

    @GetMapping("/{id}/players")
    public String viewTeamPlayers(@PathVariable Integer id, Model model) {
        Team team = teamService.getTeamById(id);
        List<Player> players = team.getPlayers();

        model.addAttribute("team", team);
        model.addAttribute("players", players);
        return "teams/players";
    }
}
```

4.1.2 Service

```
public interface TeamService {
    List<Team> getAllTeams();
    Team getTeamById(Integer id);
    void saveTeam(Team team);
    void deleteTeam(Integer id);
}

@Service
public class TeamServiceImpl implements TeamService {

    @Autowired
    private TeamRepository teamRepository;

    @Override
    public List<Team> getAllTeams() {
        return teamRepository.findAll();
    }

    @Override
    public Team getTeamById(Integer id) {
        Optional<Team> optional = teamRepository.findById(id);
        Team team = null;
        if (optional.isPresent()) {
            team = optional.get();
        } else {
            throw new RuntimeException(" Team not found for id :: " + id);
        }
        return team;
    }

    @Override
    @Transactional
    public void saveTeam(Team team) {
        this.teamRepository.save(team);
    }

    @Override
    @Transactional
    public void deleteTeam(Integer id) {
        this.teamRepository.deleteById(id);
    }
}
```

4.1.3 Repository

```
@Repository
public interface TeamRepository extends JpaRepository<Team, Integer> {
    List<Team> findByIsActive(String y);
}
```

4.2 Moduł TeamRecord

Moduł TeamRecord obsługuje dane statystyczne drużyn w danym sezonie, takie jak liczba zwycięstw, porażek i bilans punktowy.

4.2.1 Service

```
public interface TeamRecordService {
    void updateRecordsAfterGame(Integer gameId);
    List<TeamRecord> getRecordsBySeasonIdOrderByWinsDesc(Integer seasonId);
}

@Service
public class TeamRecordServiceImpl implements TeamRecordService {

    @Autowired
    private TeamRecordRepository teamRecordRepository;
    @Autowired
    private GameRepository gameRepository;
    @Autowired
    private SeasonService seasonService;

    @Override
    public void updateRecordsAfterGame(Integer gameId) {

        Game game = gameRepository.findById(gameId)
                .orElseThrow(() -> new EntityNotFoundException("Game not found with id: " + gameId));

        Team home = game.getHomeTeam();
        Team away = game.getAwayTeam();
        Season season = game.getSeason();

        TeamRecord homeRecord = teamRecordRepository
                .findByTeamAndSeason(home, season)
                .orElseGet(() -> new TeamRecord(home, season, 0, 0, 0, 0, 0, 0));

        TeamRecord awayRecord = teamRecordRepository
                .findByTeamAndSeason(away, season)
                .orElseGet(() -> new TeamRecord(away, season, 0, 0, 0, 0, 0, 0));

        boolean homeWon = game.getHomeScore() > game.getAwayScore();

        if (homeWon) {
            homeRecord.incrementWins();
            homeRecord.incrementHomeWins();

            awayRecord.incrementLosses();
            awayRecord.incrementAwayLosses();
        } else {
            awayRecord.incrementWins();
            awayRecord.incrementAwayWins();

            homeRecord.incrementLosses();
            homeRecord.incrementHomeLosses();
        }

        teamRecordRepository.save(homeRecord);
        teamRecordRepository.save(awayRecord);
    }

    private double getWinPercentage(TeamRecord r) {
        int wins = r.getWins();
        int losses = r.getLosses();
        return (wins + losses) == 0 ? 0 : (double) wins / (wins + losses);
    }

    public List<TeamRecord> getRecordsBySeasonIdOrderByWinsDesc(Integer seasonId) {
        return teamRecordRepository.findBySeasonIdOrderByWinsDesc(seasonId);
    }
}
```

4.2.2 Repository

```
@Repository
public interface TeamRecordRepository extends JpaRepository<TeamRecord, Integer> {
    Optional<TeamRecord> findByTeamAndSeason(Team home, Season season);
    List<TeamRecord> findBySeason(Season season);
    List<TeamRecord> findBySeasonIdOrderByWinsDesc(Integer seasonId);
}
```

4.3 Moduł Season

Moduł Season umożliwia zarządzanie sezonami rozgrywek koszykarskich, pozwala użytkownikowi dodawać nowe sezony oraz przełączać się między nimi w interfejsie aplikacji.

4.3.1 Controller

```
@Controller
@RequestMapping("/seasons")
public class SeasonController {

    @Autowired
    private SeasonService seasonService;

    @GetMapping
    public String listSeasons(Model model) {
        model.addAttribute("listSeasons", seasonService.getAllSeasons());
        return "seasons/list";
    }

    @GetMapping("/new")
    public String showForm(Model model) {
        model.addAttribute("season", new Season());
        model.addAttribute("formTitle", "Add New Season");
        return "seasons/form";
    }

    @PostMapping("/save")
    public String saveSeason(@ModelAttribute("season") Season season, Model model) {

        if (season.getStartDate() != null && season.getStartDate().getYear() != season.getSeasonYear()) {
            model.addAttribute("errorMessage", "Start date must be in " + season.getSeasonYear());
            return "seasons/form";
        }

        seasonService.saveSeason(season);
        return "redirect:/seasons";
    }

    @GetMapping("/edit/{id}")
    public String editSeason(@PathVariable Integer id, Model model) {
        model.addAttribute("season", seasonService.getSeasonById(id));
        model.addAttribute("formTitle", "Edit Season");
        return "seasons/form";
    }

    @PostMapping("/delete/{id}")
    public String deleteSeason(@PathVariable Integer id) {
        seasonService.deleteSeason(id);
        return "redirect:/seasons";
    }
}
```

4.3.2 Service

```
public interface SeasonService {
    List<Season> getAllSeasons();
    Season getSeasonById(Integer id);
    void saveSeason(Season season);
    void deleteSeason(Integer id);
    List<Season> findAll();
    Integer getCurrentSeasonId();
}

@Service
public class SeasonServiceImpl implements SeasonService {

    @Autowired
    private SeasonRepository seasonRepository;
    @Autowired
    private TeamRecordRepository teamRecordRepository;
    @Autowired
    private TeamRepository teamRepository;
    @Autowired
    private PlayerRepository playerRepository;
    @Autowired
    private PlayerTeamRepository playerTeamRepository;

    @Override
    public List<Season> getAllSeasons() {
        return seasonRepository.findAll();
    }

    @Override
```

```
    public Season getSeasonById(Integer id) {
        return seasonRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Season not found"));
    }

    @Override
    @Transactional
    public void saveSeason(Season season) {
        Season savedSeason = seasonRepository.save(season);

        List<Team> activeTeams = teamRepository.findByIsActive("Y");

        for (Team team : activeTeams) {
            TeamRecord record = new TeamRecord(team, savedSeason, 0, 0, 0, 0, 0, 0);
            teamRecordRepository.save(record);
        }

        List<Player> activePlayers = playerRepository.findAllByTeamInAndIsActive(activeTeams, "Y");
        for (Player player : activePlayers) {
            PlayerTeam pt = new PlayerTeam();
            pt.setPlayer(player);
            pt.setTeam(player.getTeam());
            pt.setSeasonId(savedSeason.getId());
            pt.setJerseyNumber(player.getJerseyNumber());
            pt.setStartDate(LocalDate.now());
            playerTeamRepository.save(pt);
        }
    }

    @Override
    public void deleteSeason(Integer id) {
        seasonRepository.deleteById(id);
    }

    public List<Season> findAll() {
        return seasonRepository.findAll();
    }

    public Integer getCurrentSeasonId() {
        return seasonRepository.findTopByOrderBySeasonYearDesc()
                .map(Season::getId)
                .orElseThrow(() -> new RuntimeException("No season found"));
    }
}
```

4.3.3 Repository

```
@Repository
public interface SeasonRepository extends JpaRepository<Season, Integer> {
    Optional<Season> findTopByOrderBySeasonYearDesc();
}
```

4.4 Moduł Player

Moduł Player obsługuje dane zawodników w systemie, pozwala na pełne zarządzanie zawodnikami oraz ich widoczność na liście wszystkich graczy i w kontekście konkretnej drużyny.

4.4.1 Controller

```
@Controller
@RequestMapping("/players")
public class PlayerController {

    @Autowired
    private PlayerService playerService;

    @Autowired
    private TeamService teamService;

    @Autowired
    private SeasonService seasonService;

    @Autowired
    private PlayerStatsViewService playerStatsViewService;

    @GetMapping
    public String listPlayers(Model model) {
        List<Player> players = playerService.getAllPlayers();
        model.addAttribute("players", players);
        return "players/list";
    }

    @GetMapping("/new")
```

```
    public String showAddForm(Model model) {
        model.addAttribute("player", new Player());
        model.addAttribute("teams", teamService.getAllTeams());
        return "players/form";
    }

    @PostMapping("/save")
    public String savePlayer(@Valid @ModelAttribute("player") Player player,
                             BindingResult bindingResult,
                             Model model) {
        if (bindingResult.hasErrors()) {
            model.addAttribute("teams", teamService.getAllTeams());
            return "players/form";
        }
        playerService.savePlayer(player);
        return "redirect:/players";
    }

    @GetMapping("/edit/{id}")
    public String showEditForm(@PathVariable("id") Integer id, Model model) {
        Player player = playerService.getPlayerById(id);
        model.addAttribute("player", player);
        model.addAttribute("teams", teamService.getAllTeams());
        return "players/form";
    }

    @PostMapping("/delete/{id}")
    public String deletePlayer(@PathVariable("id") Integer id) {
        playerService.deletePlayer(id);
        return "redirect:/players";
    }

    @GetMapping("/{id}/career")
    public String viewCareer(@PathVariable Integer id, Model model) {
        Player player = playerService.getById(id);
        Map<Integer, Season> seasonMap = seasonService.findAll()
                .stream().collect(Collectors.toMap(Season::getId, Function.identity()));
        List<PlayerStatsView> careerStats = playerStatsViewService.getByPlayerId(id);

        model.addAttribute("player", player);
        model.addAttribute("seasonMap", seasonMap);
        model.addAttribute("careerStats", careerStats);
        return "players/career";
    }
}
```

4.4.2 Service

```
public interface PlayerService {
    List<Player> getAllPlayers();
    Player getPlayerById(Integer id);
    void savePlayer(Player player);
    void deletePlayer(Integer id);
    Player getById(Integer id);
}

@Service
public class PlayerServiceImpl implements PlayerService {

    @Autowired
    private PlayerRepository playerRepository;
    @Autowired
    private PlayerTeamRepository playerTeamRepository;
    @Autowired
    private SeasonService seasonService;

    @PersistenceContext
    private EntityManager entityManager;

    @Override
    public List<Player> getAllPlayers() {
        return playerRepository.findAll();
    }

    @Override
    public Player getPlayerById(Integer id) {
        Optional<Player> optional = playerRepository.findById(id);
        if (optional.isPresent()) {
            return optional.get();
        } else {
            throw new RuntimeException("Player not found for id :: " + id);
        }
    }

    @Override
    @Transactional
    public void savePlayer(Player player) {
```

```
        boolean isNew = (player.getId() == null);
        Player savedPlayer;

        if (isNew) {
            savedPlayer = playerRepository.save(player);

            PlayerTeam newEntry = new PlayerTeam();
            newEntry.setPlayer(savedPlayer);
            newEntry.setTeam(savedPlayer.getTeam());
            newEntry.setSeasonId(seasonService.getCurrentSeasonId());
            newEntry.setJerseyNumber(savedPlayer.getJerseyNumber());
            newEntry.setStartDate(LocalDate.now());
            playerTeamRepository.save(newEntry);

        } else {
            Player existingPlayer = playerRepository.findById(player.getId())
                    .orElseThrow(() -> new RuntimeException("Player not found with id: " + player.getId()));

            boolean changedTeam = existingPlayer.getTeam() == null ||
                    !Objects.equals(existingPlayer.getTeam().getId(), player.getTeam().getId());

            existingPlayer.setFirstName(player.getFirstName());
            existingPlayer.setLastName(player.getLastName());
            existingPlayer.setIsActive(player.getIsActive());
            existingPlayer.setBirthDate(player.getBirthDate());
            existingPlayer.setHeight(player.getHeight());
            existingPlayer.setWeight(player.getWeight());
            existingPlayer.setNationality(player.getNationality());
            existingPlayer.setPosition(player.getPosition());
            existingPlayer.setJerseyNumber(player.getJerseyNumber());
            existingPlayer.setTeam(player.getTeam());

            savedPlayer = playerRepository.save(existingPlayer);

            if (changedTeam) {
                PlayerTeam lastTeam = playerTeamRepository.findTopByPlayerIdOrderByStartDateDesc(savedPlayer.getId());
                if (lastTeam != null) {
                    lastTeam.setEndDate(LocalDate.now());
                    playerTeamRepository.save(lastTeam);
                }

                PlayerTeam newEntry = new PlayerTeam();
                newEntry.setPlayer(savedPlayer);
                newEntry.setTeam(savedPlayer.getTeam());
                newEntry.setSeasonId(seasonService.getCurrentSeasonId());
                newEntry.setJerseyNumber(savedPlayer.getJerseyNumber());
                newEntry.setStartDate(LocalDate.now());
                playerTeamRepository.save(newEntry);
            }
        }
    }
    @Override
    @Transactional
    public void deletePlayer(Integer id) {
        playerRepository.deleteById(id);
    }

    @Override
    public Player getById(Integer id) {
        return playerRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Player not found with id: " + id));
    }
}
```

4.4.3 Repository

```
@Repository
public interface PlayerRepository extends JpaRepository<Player, Integer> {
    List<Player> findByTeamId(Integer teamId);
    List<Player> findAllByTeamInAndIsActive(List<Team> teams, String isActive);
}
```

4.5 Moduł PlayerTeam

Moduł PlayerTeam odpowiada za relację pomiędzy zawodnikiem a drużyną, odwzorowując przynależność gracza do zespołu w konkretnym sezonie.

4.5.1 Repository

```
@Repository
public interface PlayerTeamRepository extends JpaRepository<PlayerTeam, Integer>  {
    PlayerTeam findTopByPlayerIdOrderByStartDateDesc(Integer playerId);
}
```

4.6 Moduł Game

Moduł Game odpowiada za zarządzanie meczami w lidze, w tym rejestrację nowych spotkań, ich statusy oraz przechowywanie wyników.

4.6.1 Controller

```
@Controller
@RequestMapping("/games")
public class GameController {

    @Autowired
    private GameService gameService;
    @Autowired
    private TeamService teamService;
    @Autowired
    private SeasonService seasonService;
    @Autowired
    private PlayerRepository playerRepository;
    @Autowired
    private GameStatsRepository gameStatsRepository;
    @Autowired
    private TeamRecordService teamRecordService;

    @GetMapping("/{id}")
    public String getGameDetails(@PathVariable Integer id, Model model) {
        Game game = gameService.getGameById(id)
                .orElseThrow(() -> new RuntimeException("Game not found with id: " + id));
        model.addAttribute("game", game);
        return "game-details";
    }

    @GetMapping("/add")
    public String showAddGameForm(Model model) {
        model.addAttribute("game", new Game());
        model.addAttribute("teams", teamService.getAllTeams());
        model.addAttribute("seasons", seasonService.getAllSeasons());
        return "games/add-game";
    }

    @PostMapping("/add")
    public String addGame(@ModelAttribute("game") Game game, BindingResult bindingResult, Model model) {
        if (game.getHomeTeam() != null && game.getAwayTeam() != null &&
                Objects.equals(game.getHomeTeam().getId(), game.getAwayTeam().getId())) {
            bindingResult.rejectValue("awayTeam", "error.game", "Home and away teams must be different.");
        }

        if (game.getSeason() != null && game.getGameDate() != null) {
            Season season = seasonService.getSeasonById(game.getSeason().getId());
            LocalDateTime gameDate = game.getGameDate();
            LocalDateTime seasonStart = season.getStartDate().atStartOfDay();
            LocalDateTime seasonEnd = (season.getEndDate() != null) ? season.getEndDate().atTime(LocalTime.MAX) :
seasonStart.plusYears(1);

            if (gameDate.isBefore(seasonStart) || gameDate.isAfter(seasonEnd)) {
                bindingResult.rejectValue("gameDate", "error.game", "Games date must be within season.");
            }
        }

        if (bindingResult.hasErrors()) {
            model.addAttribute("teams", teamService.getAllTeams());
            model.addAttribute("seasons", seasonService.getAllSeasons());
            return "games/add-game";
        }

        gameService.saveGame(game);
        return "redirect:/";
    }

    @GetMapping("/stats/{id}")
    public String showGameStatsPage(@PathVariable Integer id, Model model) {
        Game game = gameService.getGameById(id)
                .orElseThrow(() -> new RuntimeException("Game not found with id: " + id));

        List<Player> homePlayers = playerRepository.findByTeamId(game.getHomeTeam().getId());
        List<Player> awayPlayers = playerRepository.findByTeamId(game.getAwayTeam().getId());

        Map<Integer, GameStats> statsMap;
        if ("S".equals(game.getStatus())) {
            statsMap = new HashMap<>();
        } else {
            statsMap = gameStatsRepository.findByGameId(game.getId()).stream()
                    .collect(Collectors.toMap(stat -> stat.player.getId(), stat -> stat));
        }

        model.addAttribute("game", game);
```

```
        model.addAttribute("homePlayers", homePlayers);
        model.addAttribute("awayPlayers", awayPlayers);
        model.addAttribute("statsMap", statsMap);

        model.addAttribute("statNames", Arrays.asList("rebounds", "assists", "steals", "blocks", "turnovers", "fouls"));

        return "game-stats";
    }

    @GetMapping("/live/{id}")
    public String redirectToStatsPage(@PathVariable Integer id) {
        return "redirect:/games/stats/" + id;
    }

    @PostMapping("/live/update-stat")
    public String updateStat(@RequestParam Integer gameId,
                             @RequestParam Integer playerId,
                             @RequestParam String statName,
                             @RequestParam int change) {
        gameService.updatePlayerStat(gameId, playerId, statName, change);
        return "redirect:/games/stats/" + gameId;
    }

    @PostMapping("/start/{id}")
    public String startGame(@PathVariable Integer id) {
        gameService.startGame(id);
        return "redirect:/games/stats/" + id;
    }

    @PostMapping("/finish/{id}")
    public String finishGame(@PathVariable Integer id) {
        gameService.finishGame(id);
        teamRecordService.updateRecordsAfterGame(id);
        return "redirect:/";
    }

    @PostMapping("/delete/{id}")
    public String deleteGame(@PathVariable Integer id) {
        gameService.deleteGameById(id);
        return "redirect:/";
    }
}
```

4.6.2 Service

```
public interface GameService {
    List<Game> getAllGames();
    Optional<Game> getGameById(Integer id);
    void saveGame(Game game);
    void deleteGameById(Integer id);
    void startGame(Integer gameId);
    void finishGame(Integer gameId);
    void updatePlayerStat(Integer gameId, Integer playerId, String statName, int change);
}

@Service
public class GameServiceImpl implements GameService {

    @Autowired
    private GameRepository gameRepository;
    @Autowired
    private PlayerRepository playerRepository;
    @Autowired
    private GameStatsRepository gameStatsRepository;
    @Autowired
    private TeamRecordRepository teamRecordRepository;

    @Override
    public List<Game> getAllGames() {
        return gameRepository.findAll();
    }

    @Override
    public Optional<Game> getGameById(Integer id) {
        return gameRepository.findById(id);
    }

    @Override
    public void saveGame(Game game) {
        gameRepository.save(game);
    }

    @Transactional
    @Override
    public void deleteGameById(Integer id) {
        Game game = gameRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Game not found"));
```

```
    Team winner = game.getWinnerTeam();
    Team home = game.getHomeTeam();
    Team away = game.getAwayTeam();
    Season season = game.getSeason();

    if (winner != null && home != null && away != null) {
        boolean homeWon = winner.getId().equals(home.getId());
        boolean awayWon = winner.getId().equals(away.getId());

        Optional<TeamRecord> homeRecordOpt = teamRecordRepository.findByTeamAndSeason(home, season);
        Optional<TeamRecord> awayRecordOpt = teamRecordRepository.findByTeamAndSeason(away, season);

        if (homeRecordOpt.isPresent() && awayRecordOpt.isPresent()) {
            TeamRecord homeRecord = homeRecordOpt.get();
            TeamRecord awayRecord = awayRecordOpt.get();

            if (homeWon) {
                homeRecord.setWins(Math.max(0, homeRecord.getWins() - 1));
                homeRecord.setHomeWins(Math.max(0, homeRecord.getHomeWins() - 1));

                awayRecord.setLosses(Math.max(0, awayRecord.getLosses() - 1));
                awayRecord.setAwayLosses(Math.max(0, awayRecord.getAwayLosses() - 1));

            } else if (awayWon) {
                awayRecord.setWins(Math.max(0, awayRecord.getWins() - 1));
                awayRecord.setAwayWins(Math.max(0, awayRecord.getAwayWins() - 1));

                homeRecord.setLosses(Math.max(0, homeRecord.getLosses() - 1));
                homeRecord.setHomeLosses(Math.max(0, homeRecord.getHomeLosses() - 1));
            }

            teamRecordRepository.save(homeRecord);
            teamRecordRepository.save(awayRecord);
        }
    }

    gameRepository.deleteById(id);
}

@Override
@Transactional
public void startGame(Integer gameId) {
    Game game = gameRepository.findById(gameId)
            .orElseThrow(() -> new RuntimeException("Game not found with id: " + gameId));

    if (!"S".equals(game.getStatus())) {
        throw new IllegalStateException("Game can only be started if it is scheduled.");
    }

    game.setStatus("A");

    List<Player> homePlayers = playerRepository.findByTeamId(game.getHomeTeam().getId());
    List<Player> awayPlayers = playerRepository.findByTeamId(game.getAwayTeam().getId());

    for (Player player : homePlayers) {
        if (gameStatsRepository.findByGameAndPlayer(game, player).isEmpty()) {
            gameStatsRepository.save(new GameStats(game, player, game.getHomeTeam()));
        }
    }
    for (Player player : awayPlayers) {
        if (gameStatsRepository.findByGameAndPlayer(game, player).isEmpty()) {
            gameStatsRepository.save(new GameStats(game, player, game.getAwayTeam()));
        }
    }
    gameRepository.save(game);
}

@Override
@Transactional
public void finishGame(Integer gameId) {
    Game game = gameRepository.findById(gameId)
            .orElseThrow(() -> new RuntimeException("Game not found with id: " + gameId));

    if (!"A".equals(game.getStatus())) {
        throw new IllegalStateException("Game can only be finished if it is active.");
    }

    game.setStatus("F");

    if (game.getHomeScore() > game.getAwayScore()) {
        game.setWinnerTeam(game.getHomeTeam());
    } else if (game.getAwayScore() > game.getHomeScore()) {
        game.setWinnerTeam(game.getAwayTeam());
    } else {
        game.setWinnerTeam(null);
    }
    gameRepository.save(game);
}
```

```
    @Override
    @Transactional
    public void updatePlayerStat(Integer gameId, Integer playerId, String statName, int change) {
        Game game = gameRepository.findById(gameId).orElseThrow(() -> new RuntimeException("Game not found"));
        Player player = playerRepository.findById(playerId).orElseThrow(() -> new RuntimeException("Player not found"));

        GameStats stats = gameStatsRepository.findByGameAndPlayer(game, player)
                .orElseGet(() -> new GameStats(game, player, player.getTeam()));

        switch (statName) {
            case "points":
                if (stats.points + change < 0) break;
                stats.points += change;
                if (stats.team.getId().equals(game.getHomeTeam().getId())) {
                    game.setHomeScore(game.getHomeScore() + change);
                } else {
                    game.setAwayScore(game.getAwayScore() + change);
                }

                if (change > 0) {
                    stats.fieldGoalsMade += 1;
                    stats.fieldGoalsAttempted += 1;
                    if (change == 3) {
                        stats.threePointsMade += 1;
                        stats.threePointsAttempted += 1;
                    } else if (change == 1) {
                        stats.freeThrowsMade += 1;
                        stats.freeThrowsAttempted += 1;
                    }
                } else if (change < 0) {
                    stats.fieldGoalsMade -= 1;
                    stats.fieldGoalsAttempted -= 1;
                    if (change == -3) {
                        stats.threePointsMade -= 1;
                        stats.threePointsAttempted -= 1;
                    } else if (change == -1) {
                        stats.freeThrowsMade -= 1;
                        stats.freeThrowsAttempted -= 1;
                    }
                }
                break;

            case "2p_miss":
                stats.fieldGoalsAttempted += change;
                break;
            case "3p_miss":
                stats.fieldGoalsAttempted += change;
                stats.threePointsAttempted += change;
                break;
            case "ft_miss":
                stats.freeThrowsAttempted += change;
                break;

            case "rebounds":
                if (stats.rebounds + change >= 0) stats.rebounds += change;
                break;
            case "assists":
                if (stats.assists + change >= 0) stats.assists += change;
                break;
            case "steals":
                if (stats.steals + change >= 0) stats.steals += change;
                break;
            case "blocks":
                if (stats.blocks + change >= 0) stats.blocks += change;
                break;
            case "turnovers":
                if (stats.turnovers + change >= 0) stats.turnovers += change;
                break;
            case "fouls":
                int newFouls = stats.fouls + change;
                if (newFouls >= 0 && newFouls <= 5) {
                    stats.fouls = newFouls;
                }
                break;
            default:
                throw new IllegalArgumentException("Unknown stat: " + statName);
        }

        gameStatsRepository.save(stats);
        gameRepository.save(game);
    }
}
```

4.6.3 Repository

```
public interface GameRepository extends JpaRepository<Game, Integer> {
}
```

4.7 Moduł GameStats

Moduł GameStats odpowiada za przechowywanie i przetwarzanie indywidualnych statystyk zawodników z każdego meczu.

4.7.1 Repository

```
public interface GameStatsRepository extends JpaRepository<GameStats, Integer> {
    Optional<GameStats> findByGameAndPlayer(Game game, Player player);
    List<GameStats> findByGameId(Integer gameId);
}
```

4.8 Moduł PlayerStatsView

Moduł PlayerStatsView obsługuje odczyt danych z widoku agregującego statystyki zawodników w danej drużynie i sezonie, wykorzystywany jest do prezentacji statystyk w interfejsie kariery użytkownika.

4.8.1 Service

```
public interface PlayerStatsViewService {
    List<PlayerStatsView> getByPlayerId(Integer playerId);
}

@Service
public class PlayerStatsViewServiceImpl implements PlayerStatsViewService {

    @Autowired
    private PlayerStatsViewRepository repository;

    @Override
    public List<PlayerStatsView> getByPlayerId(Integer playerId) {
        return repository.findByPlayerId(playerId);
    }
}
```

4.8.2 Repository

```
public interface PlayerStatsViewRepository extends JpaRepository<PlayerStatsView, Integer> {
    List<PlayerStatsView> findByPlayerId(Integer playerId);
    List<PlayerStatsView> findBySeasonId(Integer seasonId);
    List<PlayerStatsView> findByTeamId(Integer teamId);
    List<PlayerStatsView> findByPlayerIdAndSeasonId(Integer playerId, Integer seasonId);
}
```

5. Frontend

W celu połączenia interfejsu użytkownika z danymi z bazy danych wykorzystano bibliotekę Bootstrap. Aplikacja udostępnia widoki takie jak: strona główna z meczami, lista drużyn, zawodnicy danej drużyny, pełna lista zawodników, kariera zawodnika z jego statystykami. Dostępna jest również tabela z możliwością filtrowania według sezonu oraz dedykowana zakładka do zarządzania sezonami. Przykładowe pliki oraz wygląd poniżej:

Szablon główny HTML:

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      th:fragment="layout (content, pageTitle)" lang="pl">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title th:text="${pageTitle}">My App</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH" crossorigin="anonymous">
</head>
<body>

  <header th:replace="~{fragments/header :: header}"></header>
```

```
  <main class="container mt-4 mb-5">
    <div th:replace="${content}"></div>
  </main>

  <div th:replace="~{fragments/footer :: footer}"></div>

  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js" integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz" crossorigin="anonymous"></script>
</body>
</html>
```

Strona ze statystykami meczu:

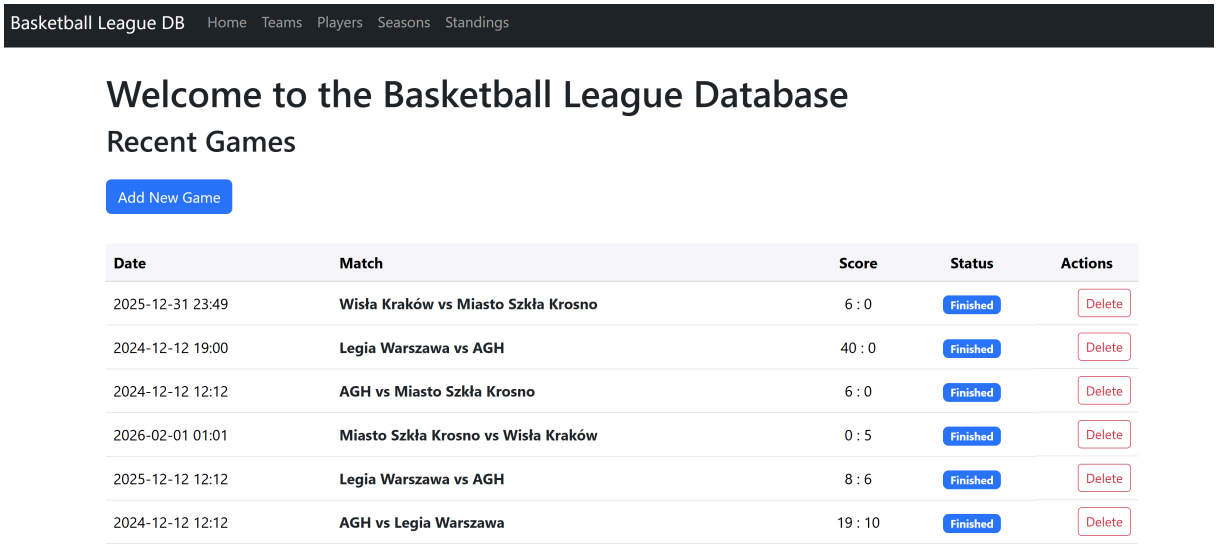
```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
      th:replace="~{layouts/main-layout :: layout(~{::content}, 'Match Statistics')}">

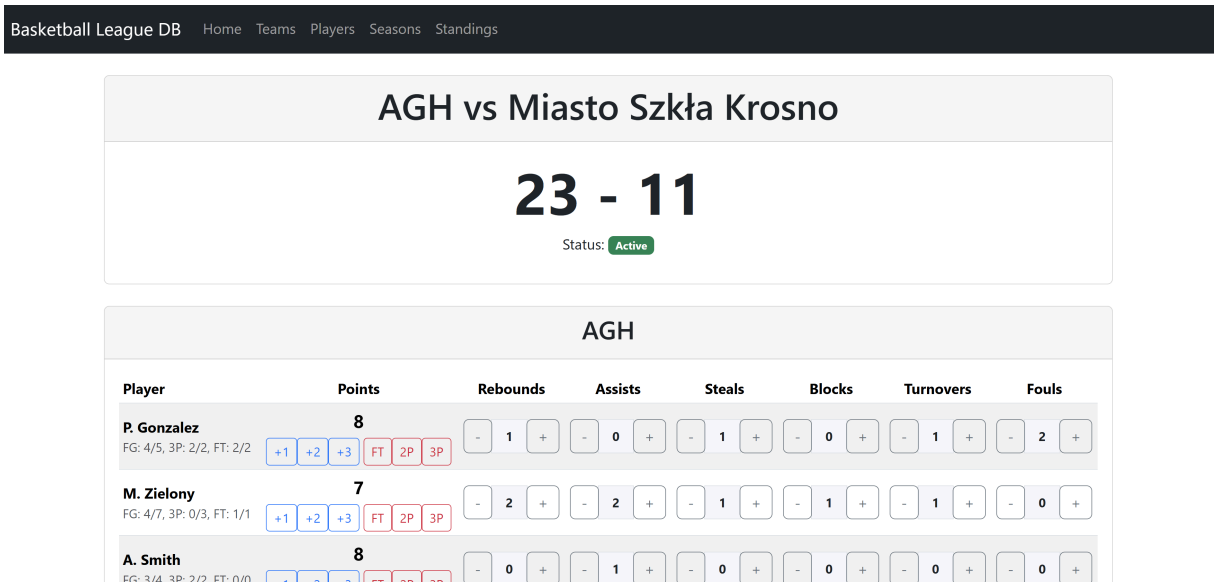
<body>
  <div th:fragment="content">
    <div class="card text-center mb-4">
      <div class="card-header">
        <h1 th:text="${game.homeTeam.name + ' vs ' + game.awayTeam.name}">Home vs Away</h1>
      </div>
      <div class="card-body">
        <h2 class="display-3 fw-bold" th:text="${game.homeScore + ' - ' + game.awayScore}">0 - 0</h2>
        <p>Status:
          <span th:if="${game.status == 'S'}" class="badge bg-secondary">Scheduled</span>
          <span th:if="${game.status == 'A'}" class="badge bg-success">Active</span>
          <span th:if="${game.status == 'F'}" class="badge bg-primary">Finished</span>
        </p>
      </div>
    </div>

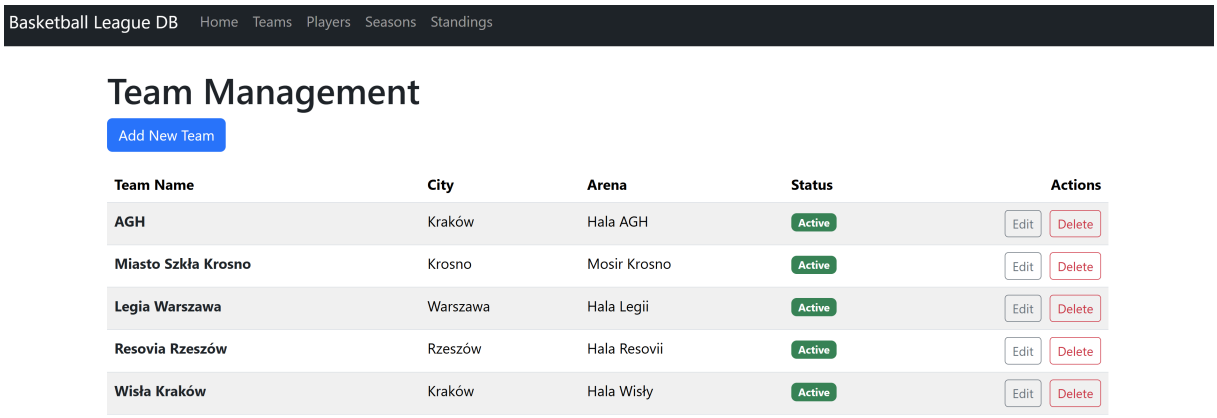
    <div class="d-grid gap-4">
      <div th:replace="~{fragments/stats-table :: stats-panel(team=${game.homeTeam}, players=${homePlayers}, game=${game},
statsMap=${statsMap})}"></div>

      <div th:replace="~{fragments/stats-table :: stats-panel(team=${game.awayTeam}, players=${awayPlayers}, game=${game},
statsMap=${statsMap})}"></div>
    </div>
  </div>
</body>
</html>
```

Frontend menu głównego, obsługi pojedynczego meczu i widoku drużyn







6. Podsumowanie

Projekt pokazuje praktyczne zastosowanie architektury Spring Boot w połączeniu z Hibernate i relacyjną bazą danych. Umożliwia zarządzanie ligą koszykarską przez przemyślaną strukturę encji, kontrolerów i widoków. System jest przejrzysty i z możliwością dalszego rozszerzenia.